

ULBRA - UNIVERSIDADE LUTERANA DO BRASIL
CURSO DE ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

SANDRA HELOISA GUINCHESKI DE SOUZA

PROGRAMAÇÃO ORIENTADA A OBJETOS (POO)

TORRES, RIO GRANDE DO SUL
2024

SUMÁRIO

1. RESUMO.....	
2. INTRODUÇÃO.....	
3. DESENVOLVIMENTO.....	
3.1. ENCAPSULAMENTO.....	
3.1.1 ATRIBUTOS PRIVADOS.....	
3.1.2 USO DE MÉTODOS DE ACESSOS (GETTERS E SETTERS).....	
3.2. HERANÇA.....	
3.3. POLIMORFISMO.....	
3.4. ABSTRAÇÃO.....	
4. CONCLUSÃO.....	
5. REFERÊNCIAS.....	

1. RESUMO

Este trabalho apresenta o desenvolvimento de um sistema de gerenciamento de biblioteca utilizando os conceitos da Programação Orientada a Objetos (POO). O sistema permite o cadastro e a manipulação de livros e usuários, incluindo funções de empréstimo e devolução de livros.

O projeto explora os quatro pilares da POO — encapsulamento, herança, polimorfismo e abstração — através da criação de classes e interfaces que organizam e modularizam o código. A classe Biblioteca gerencia o acervo de livros e o cadastro de usuários, enquanto as classes Livro e Usuario implementam o controle de estoque e exibição de detalhes, respectivamente. O uso das interfaces como IEmprestavel e IPesquisavel garantem a flexibilidade necessária no sistema.

Palavras-chave: Programação Orientada a Objetos, encapsulamento, herança, polimorfismo, abstração, sistema de biblioteca, C#, interfaces

2. INTRODUÇÃO

O desenvolvimento de sistemas de gerenciamento para bibliotecas visa otimizar o controle de acervos, empréstimos e o gerenciamento de usuários, facilitando o acesso e organização de informações. Este projeto oferece a criação de um sistema de gerenciamento para uma biblioteca que permita o cadastro, consulta e empréstimo de livros, assim como o gerenciamento de usuários, utilizando os princípios da Programação Orientada a Objetos (POO) e a linguagem C#.

A POO se destaca como uma abordagem eficaz para a construção de sistemas complexos, pois organiza o código em torno de objetos, que são instâncias de classes representando entidades do mundo real. Cada classe encapsula dados e comportamentos, o que permite a modularização e manutenção facilitada do software. Além disso, conceitos como herança, polimorfismo, e abstração aumentam a reutilização de código e a flexibilidade na implementação de novas funcionalidades.

A importância da POO neste projeto reside na capacidade de modularizar as funcionalidades do sistema, garantindo que os componentes, como o cadastro de livros e o gerenciamento de usuários, sejam implementados de forma independente, mas interconectada. Ao longo do trabalho, serão apresentados exemplos práticos e aplicação dos quatro pilares da POO — encapsulamento, herança, polimorfismo e abstração — no desenvolvimento do sistema, com trechos em código C#.

3. DESENVOLVIMENTO

3.1 ENCAPSULAMENTO

O encapsulamento é o princípio da programação orientada a objetos que agrupa os dados e os métodos que os manipulam em uma única entidade, escondendo os detalhes internos da implementação. Esse conceito permite que apenas os métodos públicos de um objeto possam acessar ou alterar seus atributos, evitando interferências indevidas e facilitando a manutenção.

A principal vantagem do encapsulamento em relação à programação estruturada é que ele oculta a complexidade dos objetos, permitindo que você use suas funcionalidades sem precisar entender como elas funcionam por dentro. Um exemplo que pode ser mencionado é o telefone, ao discar um número, não é necessário saber o que acontece por trás para estabelecer a conexão.

Outra vantagem é a flexibilidade de modificar ou expandir a funcionalidade interna de um objeto sem impactar as partes do sistema que o utilizam. Assim como uma companhia telefônica pode atualizar sua tecnologia sem mudar a forma como você faz uma ligação, um objeto encapsulado pode evoluir sem alterar a maneira como os outros interagem com ele.

O essencial para essa interação é que um objeto conheça a interface do outro, ou seja, o conjunto de operações disponíveis para acesso.

3.1.1 Atributos Privados

No código, o encapsulamento pode ser perceptível na classe `Livro`, onde os atributos `quantidadeEstoque` e `emprestado` são declarados como privados. Assim, eles não podem ser acessados diretamente fora da classe pela sua restrição direta aos dados:

```
5 | 7 references  
   | private int quantidadeEstoque;  
6 | 6 references  
   | private bool emprestado;
```

3.1.2 Uso de métodos de acesso (getters e setters)

Os atributos isbn, genero e quantidadeEstoque são privados, é fornecido métodos públicos ou propriedades que disponibilizam o acesso a eles, como no caso da propriedade QuantidadeEmEstoque. Esse método público permite que os dados internos sejam manipulados, mantendo a integridade do mesmo.

```

8      public int QuantidadeEmEstoque
9      {
10         get { return quantidadeEstoque; }
11         set { quantidadeEstoque = value; }
12     }
13

```

A propriedade QuantidadeEmEstoque é usada para acessar o valor de quantidadeEstoque de forma controlada. Isso é uma prática comum de encapsulamento, onde você oculta o campo privado e controla seu acesso através de métodos públicos.

3.1.3 Métodos que manipulam o estado de um objeto

Os métodos Emprestar e Devolver são exemplos de encapsulamento pois manipulam os atributos internos: emprestado e quantidadeEstoque sem permitir acesso direto a esses atributos fora da classe. Quem usa o objeto Livro não precisa saber como esses atributos são atualizados, apenas interage com os métodos da classe.

```

32      1 reference
33      public void Emprestar(Usuario usuario)
34      {
35          if (quantidadeEstoque > 0 && !emprestado)
36          {
37              emprestado = true;
38              quantidadeEstoque--;
39              Console.WriteLine($"Livro '{Titulo}' emprestado para {usuario.Nome}.");
40          }
41          else
42          {
43              Console.WriteLine("O livro não está disponível para empréstimo.");
44          }
45      }
46
47      1 reference
48      public void Devolver()
49      {
50          if (emprestado)
51          {
52              emprestado = false;
53              quantidadeEstoque++;
54              Console.WriteLine($"Livro '{Titulo}' devolvido.");
55          }
56          else
57          {
58              Console.WriteLine("O livro não está emprestado.");
59          }
60      }

```

3.2. HERANÇA

Em programação orientada a objetos, a herança permite que uma classe herde características (atributos e métodos) de outra classe, facilitando a reutilização e organização do código. Assim como na genética, onde características dos pais são transmitidas para os filhos, na herança entre classes, uma classe “filha” herda propriedades da classe “pai” (que seria esse, uma superclasse). Isso significa que os métodos e atributos da superclasse são disponíveis de forma automática e garantida, para as subclasses (seus filhos), permitindo que a nova classe aproveite o código existente e apenas adicione ou modifique funcionalidades específicas. Por exemplo, uma classe “Mamífero” pode ter atributos como “Possui pelos” e “mamar”. Uma classe “Primata” pode herdar esses atributos e adicionar novos, como “Possui mãos e pés”. Já uma classe “Ser Humano”, que herda de “Primata”, terá acesso tanto aos atributos de “Mamífero” quanto aos de “Primata”, além de poder adicionar métodos próprios, como “Andar Ereto” e “Filosofar”. Essa estrutura de herança permite que as classes compartilhem funcionalidades sem a necessidade de duplicação de código, e quando uma mudança é feita na superclasse, todas as subclasses automaticamente utilizam essa modificação.

Essa prática não só economiza código como também mantém a integridade do sistema, garantindo que mudanças em uma classe base sejam refletidas em todas as suas descendentes.

No projeto, a herança foi utilizada na criação da classe `ItemBiblioteca`, servindo como classe base para `Livro`. A classe `ItemBiblioteca` encapsula os atributos e métodos comuns a todos os itens de uma biblioteca, como título e autor, permitindo que classes derivadas, como `Livro`, herdem esses atributos e comportamentos.

ItemBiblioteca.cs > ...

```
1 public abstract class ItemBiblioteca
2 {
3     8 references
4     public string Titulo { get; set; }
5     3 references
6     public string Autor { get; set; }
7
8     1 reference
9     public ItemBiblioteca(string titulo, string autor)
10    {
11        Titulo = titulo;
12        Autor = autor;
13    }
14
15    2 references
16    public virtual void ExibirInformacoes()
17    {
18        Console.WriteLine("Título: " + Titulo);
19        Console.WriteLine("Autor: " + Autor);
20    }
21 }
```

```

Livro.cs > Livro
32 references
1 public class Livro : ItemBiblioteca, IEmprestavel, IPesquisavel
2 {
3     3 references
4     public string isbn { get; private set; }
5     3 references
6     public string Genero { get; private set; }
7     7 references
8     private int quantidadeEstoque;
9     6 references
10    private bool emprestado;
11
12    5 references
13    public int QuantidadeEmEstoque
14    {
15        get { return quantidadeEstoque; }
16        set { quantidadeEstoque = value; }
17    }
18
19    7 references
20    public Livro(string titulo, string autor, string isbn, string genero, int quantidadeEstoque)
21    : base(titulo, autor)
22    {
23        this.isbn = isbn;
24        this.Genero = genero;
25        this.quantidadeEstoque = quantidadeEstoque;
26        this.emprestado = false;
27    }
28
29    2 references
30    public override void ExibirInformacoes()
31    {
32        base.ExibirInformacoes();
33        Console.WriteLine("ISBN: " + isbn);
34        Console.WriteLine("Gênero: " + Genero);
35        Console.WriteLine("Quantidade em estoque: " + quantidadeEstoque);
36        Console.WriteLine("Status do empréstimo: " + (emprestado ? "Emprestada" : "Disponível"));
37    }
38 }

```

Nesse exemplo, a classe Livro herda de ItemBiblioteca, reutilizando atributos e métodos comuns e, ao mesmo tempo, estendendo as funcionalidades com novos atributos, como ISBN e Genero, além de sobrescrever o método ExibirInformacoes.

3.3 POLIMORFISMO

O polimorfismo é a capacidade de uma variável de se referir, em tempo de execução, a objetos de diferentes classes. Essa característica permite que objetos distintos respondam a uma mesma mensagem, apresentando formas de execução específicas para cada um. Assim, o usuário pode enviar uma mensagem genérica e ignorar os detalhes da implementação do objeto receptor.

O polimorfismo se divide em duas características principais: polimorfismo em tempo de compilação, por meio da sobrecarga de métodos, e polimorfismo em tempo de execução, por meio da sobrescrição de métodos.

É fundamental para a flexibilidade do código, pois permite que métodos e funções

sejam implementados de forma genérica, podendo operar em diversos tipos de objetos. Essa característica é utilizada em frameworks e bibliotecas, onde a interação com diferentes tipos de dados é comum.

No projeto, o polimorfismo é apresentado na classe Interface `IEmprestavel` e na classe `Livro`:

A classe `Livro` implementa a interface `IEmprestavel`, que define métodos que podem ser utilizados em diferentes tipos de itens que podem ser emprestados.

Quando tem métodos que aceitam a interface `IEmprestavel` como parametro, podem ser passadas instâncias de qualquer classe que implemente essa interface.

```
7  public interface IEmprestavel
8  {
9      1 reference
      void Emprestar(Usuario usuario);
10     1 reference
      void Devolver();
11 }
12
```

```

Livros > Livro
32 references
1 public class Livro : ItemBiblioteca, IEmprestavel, IPesquisavel
2 {
3     3 references
4     public string Isbn { get; private set; }
5     3 references
6     public string Genero { get; private set; }
7     7 references
8     private int quantidadeEstoque;
9     6 references
10    private bool emprestado;
11
12    5 references
13    public int QuantidadeEmEstoque...
14
15    7 references
16    public Livro(string titulo, string autor, string isbn, string genero, int quantidadeEstoque) ...
17
18    2 references
19    public override void ExibirInformacoes()...
20
21    1 reference
22    public void Emprestar(Usuario usuario)
23    {
24        if (quantidadeEstoque > 0 && !emprestado)
25        {
26            emprestado = true;
27            quantidadeEstoque--;
28            Console.WriteLine($"Livro '{Titulo}' emprestado para {usuario.Nome}.");
29        }
30        else
31        {
32            Console.WriteLine("O livro não está disponível para empréstimo.");
33        }
34    }
35
36    1 reference
37    public void Devolver()
38    {
39        if (emprestado)
40        {
41            emprestado = false;
42            quantidadeEstoque++;
43            Console.WriteLine($"Livro '{Titulo}' devolvido.");
44        }
45        else
46        {
47            Console.WriteLine("O livro não está emprestado.");
48        }
49    }
50
51    }
52
53    }
54
55    }
56
57    }
58
59    }
60
61    }
62
63    }
64
65    }
66
67    }
68
69    }
70
71    }
72
73    }
74
75    }
76
77    }
78
79    }
80
81    }
82
83    }
84
85    }
86
87    }
88
89    }
90
91    }
92
93    }
94
95    }
96
97    }
98
99    }
100   }

```

ecaOOP (BibliotecaOOP) Projects: BibliotecaOOP Debug Any CPU

3.4 ABSTRAÇÃO

A abstração é um dos pilares da POO e desempenha um papel importante na forma como os desenvolvedores modelam os sistemas complexos. Esse conceito se refere à capacidade de simplificar a complexidade do mundo real ao se concentrar nas características essenciais de um objeto, enquanto oculta detalhes desnecessários ou irrelevantes. Ao aplicar a abstração, os programadores podem criar modelos que representam entidades do mundo real de maneira gerenciável e compreensível, ou seja, na programação orientada a objetos, os desenvolvedores podem criar classes que servem como moldes para criar objetos, enquanto essas classes descrevem os atributos (propriedades) e comportamentos (métodos) comuns de um grupo de objetos, fornecendo uma interface clara para a interação com eles. Por exemplo, em um sistema que gerencia uma biblioteca, pode uma classe abstrata `ItemBiblioteca` que define atributos como `Titulo` e `Autor`, bem como o método `ExibirInformacoes()`. Essa classe serve como um modelo para subclasses

específicas, como Livro.



```
ItemBiblioteca.cs > ...  
3 references  
1 public abstract class ItemBiblioteca  
2 {  
3     8 references  
4     public string Titulo { get; set; }  
5     3 references  
6     public string Autor { get; set; }  
7  
8     1 reference  
9     public ItemBiblioteca(string titulo, string autor)  
10    {  
11        Titulo = titulo;  
12        Autor = autor;  
13    }  
14  
15    2 references  
16    public virtual void ExibirInformacoes()  
17    {  
18        Console.WriteLine("Título: " + Titulo);  
19        Console.WriteLine("Autor: " + Autor);  
20    }  
21 }
```

4. CONCLUSÃO

O desenvolvimento do sistema de gerenciamento de biblioteca, fundamentado nos princípios da Programação Orientada a Objetos (POO), demonstrou a eficácia e a relevância dessa abordagem na criação de softwares robustos, escaláveis e flexíveis. Por meio da implementação de encapsulamento, herança, polimorfismo e abstração, foi possível estruturar o código de forma modular, garantindo manutenção simplificada, reutilização eficaz e facilidade na expansão de funcionalidades.

O encapsulamento foi essencial para proteger os dados internos das classes, permitindo interações seguras e controladas entre os objetos e preservando a integridade dos dados. A herança, por sua vez, facilitou a extensão e a organização do código, promovendo a reutilização de atributos e métodos comuns entre diferentes tipos de itens, reduzindo a duplicação de código e aumentando a coesão. O polimorfismo trouxe flexibilidade ao sistema, possibilitando que diferentes tipos de itens fossem tratados de maneira uniforme, o que simplificou a lógica do gerenciamento de empréstimos e contribuiu para a adaptabilidade do software. Por fim, a abstração reduziu a complexidade do sistema ao

focar nos aspectos essenciais dos objetos, permitindo uma modelagem mais clara, intuitiva e alinhada aos requisitos do domínio da aplicação.

Este trabalho não apenas ilustra a aplicação prática dos conceitos da POO, mas também evidencia a importância de um planejamento cuidadoso na definição das classes e interfaces, que são fundamentais para a criação de um projeto bem estruturado e de fácil manutenção. A experiência adquirida ao longo do desenvolvimento deste sistema serve como base sólida para projetos futuros, demonstrando como a implementação dos pilares da POO pode contribuir significativamente para a criação de softwares mais eficientes, flexíveis e duradouros.

Assim, conclui-se que o domínio da Programação Orientada a Objetos é indispensável para a construção de sistemas complexos e de qualidade, reforçando a importância de se continuar explorando e aprimorando essas habilidades em projetos subsequentes. A abordagem orientada a objetos não apenas facilita o desenvolvimento, mas também proporciona um framework poderoso para enfrentar desafios futuros, garantindo que os sistemas sejam mais adaptáveis às mudanças e evoluções das necessidades dos usuários.

5. REFERÊNCIAS

FARINELLI, Fernanda. Conceitos básicos de programação orientada a objetos. **Instituto Federal Sudeste de Minas Gerais**, 2007.

RICARTE, Ivan Luiz Marques. Programação Orientada a Objetos: uma abordagem com Java. <http://www.dca.fee.unicamp.br/cursos/PooJava/Aulas/poojava.pdf>> Acesso em, v. 29, n. 10, p. 2014, 2001.

JUNGTHON, Gustavo; GOULART, Cristian Machado. Paradigmas de Programação. **Monografia (Monografia)—Faculdade de Informática de Taquara, Rio Grande do Sul**, v. 57, 2009.